CLASSICAL MACHINE LEARNING

CHAPTER 54

Machine Learning Quick Reference

Algorithms, key formulas, metrics and code snippets

Revision sheet for your soutenance



Contents

Chapter 54 — Final Revision Cheat Sheets **1**
The ML lifecycle in 12 verbs . . . 1
Task-to-algorithm lookup . . . 1
Key equations — regression . . . 1
Key equations — classification . . . 2
Key equations — unsupervised . . . 2
Model parameters that examiners ask about . . . 2
Quick, safe Python recipes . . . 3
Error-pattern quick diagnosis . . . 4
10 sentences worth memorizing . . . 4
90-second exam readiness checklist . . . 4

Chapter 54 — Final Revision Cheat Sheets

For the final hour before a soutenance croisée. Detailed explanations are in Parts I–VII. Key answer format: *What is it? → How does it work? → When use it? → What could go wrong? → How would I evaluate?*

The ML lifecycle in 12 verbs

Define → Audit → Split → Explore → Engineer → Pipeline → Baseline → Compare → Tune → Diagnose → Test → Deploy/Monitor.

Gold rule: The model, preprocessing, feature selection, resampling and threshold must never learn from test observations/labels. Group/time structure determines split strategy.

Task-to-algorithm lookup

Problem	Baseline	Main candidates	Key metric
Continuous numeric target	Dummy mean/median	Linear, Ridge, RF, SVR, boosting	MAE, RMSE, R^2
Binary label	Dummy majority/prior	Logistic, RF, SVM, XGBoost	Precision, recall, F1, AP, ROC-AUC
Multiclass label	Dummy	Multinomial logistic, RF, boosting	Macro F1, log loss, accuracy
Sparse text classification	Majority	TF-IDF + NB/LinearSVC/logistic	Macro F1, AP where relevant
Segmentation	Descriptive baselines	K-Means, GMM, DBSCAN, hierarchical	Stability, silhouette, business use
Dimensionality reduction	None	PCA, SVD; t-SNE/UMAP for visualization	Explained variance / visual structure
Unusual-case detection	Domain rule	Isolation Forest, LOF, One-Class SVM	Precision/recall at alert capacity if labels exist

Key equations — regression

$$\hat{y} = \beta_0 + \sum_j \beta_j x_j; \quad e_i = y_i - \hat{y}_i.$$

$$MAE = \tfrac{1}{n}\sum |e_i|; \quad MSE = \tfrac{1}{n}\sum e_i^2; \quad RMSE = \sqrt{MSE}.$$

$$R^2 = 1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2}; \quad R^2_{adj} = 1 - (1-R^2)\frac{n-1}{n-p-1}.$$

$$MAPE = \tfrac{1}{n}\sum |e_i / y_i| \quad \text{(unstable at/near } y_i = 0\text{)}.$$

Ridge: L2 penalty $\alpha\|\beta\|_2^2$; **Lasso:** L1 penalty $\alpha\|\beta\|_1$; **ElasticNet:** both. Coefficient meanings are conditional on included predictors and units. Ridge shrinks; Lasso can select zero weights.

Key equations — classification

$$z = w^\top x + b, \qquad P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Metric	Formula
Accuracy	$(TP + TN)/(TP + TN + FP + FN)$
Precision	$TP/(TP + FP)$
Recall / TPR	$TP/(TP + FN)$
Specificity / TNR	$TN/(TN + FP)$
FPR	$FP/(FP + TN)$
F1	$2TP/(2TP + FP + FN)$
Balanced accuracy (binary)	$(TPR + TNR)/2$
ROC-AUC	Area under TPR vs FPR threshold curve
Average precision	Precision weighted by recall increases
Binary log loss	$-\frac{1}{n}\sum[y \ln p + (1 - y) \ln(1 - p)]$

Threshold: Smaller threshold often increases recall, at cost of false positives. Choose on validation/CV, never optimize after examining final test. AUC ranking ≠ calibration. Precision is prevalence-dependent. **Label meaning**: check `model.classes_` before `[:,1]`.

Key equations — unsupervised

$$\text{Inertia} = \sum_i \|x_i - \mu_{c_i}\|^2; \quad \text{Silhouette}(i) = \frac{b_i - a_i}{\max(a_i, b_i)}.$$

K-Means: k groups, compact centroid distance; initialize with K-Means++ and multiple starts. **DBSCAN:** `eps`, `min_samples`, noise label -1, variable-density challenge. **GMM:** soft Gaussian membership, covariance matters. **PCA:** orthogonal directions maximizing **input** variance; not necessarily target predictiveness. **t-SNE/UMAP:** visualizations where distances between islands should be interpreted cautiously.

Model parameters that examiners ask about

Model	Main controls	Larger means…
KNN	`n_neighbors`	Smoother/locality less detailed
Decision Tree	`max_depth`	More splits/complexity
Random Forest	`n_estimators`	More trees, usually stabler and slower
Random Forest	`min_samples_leaf`	Smoother, fewer tiny noisy leaves
Logistic Regression	`C`	Weaker regularization
SVM RBF	`C`	Stronger penalty on margin violations
SVM RBF	`gamma`	More local nonlinear influence
Ridge/Lasso	`alpha`	Stronger shrinkage
SVR	`epsilon`	Wider loss-free tolerance tube
Boosting	`learning_rate`	Larger step each stage, potentially more overfit
Boosting	`n_estimators`	More additive stages
K-Means	`n_clusters`	More centroids, lower inertia possible
DBSCAN	`eps`	Larger density neighborhoods / merged groups

Model	Main controls	Larger means…
PCA	`n_components`	More axes and preserved variance

Quick, safe Python recipes

Classification

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=.2, random_state=42, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
print(cross_val_score(model, X_dev, y_dev, cv=5, scoring='balanced_accuracy').mean())
model.fit(X_dev, y_dev)
print(classification_report(y_test, model.predict(X_test)))
print(roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]))
```

Regression

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
Xd, Xt, yd, yt = train_test_split(X, y, test_size=.2, random_state=42)
model = make_pipeline(StandardScaler(), Ridge(alpha=1))
print((-cross_val_score(model, Xd, yd, cv=5,
                        scoring='neg_mean_absolute_error')).mean())
model.fit(Xd, yd)
pred = model.predict(Xt)
print(mean_absolute_error(yt,pred), root_mean_squared_error(yt,pred), r2_score(yt,pred))
```

Pipeline with categorical data

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
pre = ColumnTransformer([
    ('num', Pipeline([('imp', SimpleImputer(strategy='median')),
                      ('scale', StandardScaler())]), numeric_cols),
    ('cat', Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                      ('onehot', OneHotEncoder(handle_unknown='ignore'))]), categorical_cols)
])
model = Pipeline([('pre', pre), ('estimator', LogisticRegression(max_iter=2000))])
```

Cross-validation / tuning

```
from sklearn.model_selection import GridSearchCV
search = GridSearchCV(model,
    {'estimator__C':[0.1,1,10]},
    cv=5, scoring='average_precision', n_jobs=-1)
# search.fit(X_dev, y_dev)  # NEVER tune on X_test/y_test
```

Save/reload a pipeline (trusted files only)

```
import joblib
joblib.dump(model, 'complete_model.joblib')
# restored = joblib.load('complete_model.joblib')
```

Error-pattern quick diagnosis

What you observe	First suspicion	Fix/diagnostic
Accuracy 99%, recall 0	Severe imbalance	Baseline, confusion matrix, PR
Train 100%, CV 60%	Overfit / leakage / nonrepresentative folds	Audit splits, regularize
CV high, future test low	Time drift / leakage	Temporal evaluation, inspect timestamps
KNN is nonsense	Unscaled irrelevant features	Scale/train-fitted, feature selection
SVM low performance	Scaling, C/gamma	Proper pipeline and CV tuning
Tree has huge importance on ID	High cardinality / leakage	Drop/audit ID, group split
RMSE large vs MAE	Very large errors	Inspect residual tail/groups
K-Means bad segments	Wrong distance/scaling/k	Feature selection, test alternatives
Inference shape mismatch	Encoder/schema version drift	Save full pipeline, validate DataFrame schema
Probability .9 but 50% correct	Calibration problem	Reliability diagram, calibration folds

10 sentences worth memorizing

1. “The split must represent the way the model will be used.”
2. “I chose the metric based on the cost of errors.”
3. “All learned preprocessing lives inside the cross-validated pipeline.”
4. “I compare to a dummy baseline and a simple interpretable model.”
5. “I keep the final test set untouched until choices are locked.”
6. “Precision measures correctness of positive alerts; recall measures coverage of actual positives.”
7. “AUC measures discrimination; calibration measures probability reliability.”
8. “Random Forest averages diverse trees; boosting builds sequential corrections.”
9. “Clustering labels are not ground-truth categories or automatically actionable.”
10. “A useful model includes monitoring, limitations and a reproducible artifact.”

90-second exam readiness checklist

- ☐ Can I describe my target/horizon, grouping, number of rows and class balance?
- ☐ Can I show the exact pipeline and justify its encoders/scalers?
- ☐ Can I derive and interpret regression or classification metrics by hand?
- ☐ Can I explain CV model selection and final test separation?
- ☐ Can I justify why my chosen model beats simpler baselines?
- ☐ Can I state three limitations and a plan to monitor them?

For detailed corrections, see Chapter 53.